

Можно ли строить прогнозы и кластеризацию по трём таблицам?

Как корректно подготовить данные, если таблицы имеют разную гранулярность

Да, можно. Но модели обычно получают не три исходные таблицы, а одну аналитическую матрицу признаков. Сначала нужно определить объект анализа и привести все признаки к единой гранулярности.

1. Главный вопрос: что означает одна строка?

Гранулярность - это уровень детализации данных. Перед объединением таблиц необходимо зафиксировать, что представляет одна строка итогового набора. Алгоритмы кластеризации обычно принимают матрицу вида объекты × признаки, то есть одну строку на каждый анализируемый объект.^[1]

Задача	Одна строка итогового датасета	Ключ
Кластеризация клиентов	Один клиент	customer_id
Прогноз времени решения обращения	Одно обращение	ticket_id
Прогноз числа обращений	Один день или день × группа	date / date + group
Кластеризация товаров	Один товар	product_id

2. Почему нельзя просто соединить все строки

Предположим, имеются три таблицы: customers - одна строка на клиента, tickets - много обращений одного клиента, payments - много платежей одного клиента. Если напрямую соединить обращения и платежи по customer_id, возникнет связь many-to-many: каждое обращение клиента скомбинируется с каждым его платежом. В результате строки искусственно размножатся, а суммы, средние и количество событий окажутся искажены.

Пример: 4 обращения × 3 платежа = 12 строк после прямого соединения, хотя исходно было только 7 событий.

3. Правильный подход для кластеризации клиентов

Если объект кластеризации - клиент, все дополнительные таблицы нужно сначала агрегировать до уровня `customer_id`. После этого получается одна строка с характеристиками каждого клиента: число обращений, среднее время решения, доля негативных оценок, сумма платежей, средний платёж и другие признаки.

```
ticket_features = (
    tickets
    .groupby("customer_id", as_index=False)
    .agg(
        tickets_count=("ticket_id", "nunique"),
        avg_resolution_hours=("resolution_hours", "mean"),
        negative_share=("is_negative", "mean")
    )
)

payment_features = (
    payments
    .groupby("customer_id", as_index=False)
    .agg(
        payments_count=("payment_id", "nunique"),
        total_amount=("amount", "sum"),
        avg_amount=("amount", "mean")
    )
)

customer_dataset = (
    customers
    .merge(ticket_features, on="customer_id", how="left",
           validate="one_to_one")
    .merge(payment_features, on="customer_id", how="left",
           validate="one_to_one")
)
```

Параметр `validate` в `pandas.merge()` проверяет ожидаемую кардинальность ключей и помогает обнаружить неожиданное дублирование до того, как оно испортит расчёты.^[2]

После объединения обязательно проверить

1. число строк до и после каждого `merge`;
2. уникальность `customer_id` в итоговой таблице;
3. пропуски в присоединённых признаках;
4. диапазоны и единицы измерения признаков;
5. масштабирование числовых признаков перед методами, основанными на расстоянии.

Стандартизация часто необходима для алгоритмов машинного обучения: признаки с существенно разными масштабами могут непропорционально влиять на расстояния и результат кластеризации.^[5]

4. Для прогноза по каждому обращению гранулярность другая

Если требуется предсказать время решения конкретного обращения, базовой таблицей становится `tickets`. Клиентские характеристики можно присоединить как справочник по связи `many-to-one`: много обращений относятся к одному клиенту.

```
ticket_dataset = tickets.merge(
    customers,
    on="customer_id",
    how="left",
    validate="many_to_one"
)
```

Таблицу платежей при этом также нельзя присоединять в исходном виде. Нужно сформировать признаки, релевантные моменту создания обращения, например:

- количество платежей клиента до даты обращения;
- сумма платежей за предыдущие 30 дней;
- средний платёж до момента создания обращения;
- число дней с последнего платежа.

Важно: использовать можно только информацию, которая была доступна на момент прогнозирования. Признаки из будущего создают утечку данных и завышают оценку качества модели.^[3]

5. Для временного прогноза все источники приводятся к времени

Если прогнозируется число обращений, итоговая гранулярность обычно задаётся периодом: день, неделя или месяц. Таблицы агрегируются к этому периоду, после чего объединяются по дате. Дополнительно создаются лаговые и скользящие признаки. В официальном примере `scikit-learn` для временных рядов прогноз строится именно на матрице лаговых признаков, а оценивание выполняется с учётом временного порядка.^[4]

date	tickets_count	avg_rating	active_clients	marketing_cost
2026-07-01	125	4.2	91	50 000
2026-07-02	138	4.0	97	62 000

6. Универсальный алгоритм подготовки данных

Шаг	Действие	Контрольный вопрос
1	Определить объект	Что прогнозируем или кластеризуем: клиента, обращение, товар, день?
2	Зафиксировать ключ	Какой идентификатор должен быть уникальным в итоговой таблице?
3	Описать гранулярность источников	Одна строка в каждой исходной таблице означает что именно?
4	Агрегировать детали	Таблицы более низкой детализации привести к уровню объекта анализа.
5	Присоединить справочники	Использовать ожидаемые связи one-to-one или many-to-one.
6	Проверить кардинальность	Сравнить число строк, уникальные ключи и пропуски после merge.
7	Исключить утечку	Для прогноза оставить только информацию, доступную на момент предсказания.
8	Подготовить признаки	Кодирование категорий, заполнение пропусков, масштабирование, отбор признаков.

Готовая формулировка для слушателя

Три таблицы использовать можем. Объединять их нужно не обязательно в исходном виде, а через подготовку признаков на едином уровне детализации. Сначала определяем, что означает одна строка будущего датасета, затем агрегируем каждую таблицу до этого уровня и только после этого соединяем. Если гранулярность не согласовать, строки могут размножиться, показатели исказятся, а модель будет обучаться на некорректных данных.

Источники

- [1] [scikit-learn. Clustering: стандартная матрица входных данных имеет форму \(n_samples, n_features\).](#)
- [2] [pandas. DataFrame.merge и аргумент validate для проверки кардинальности ключей.](#)
- [3] [scikit-learn. Common pitfalls: рекомендации по предотвращению data leakage.](#)
- [4] [scikit-learn. Lagged features for time series forecasting.](#)
- [5] [scikit-learn. Preprocessing data: стандартизация признаков.](#)

Материал подготовлен как краткий ответ слушателю. Дата проверки источников: 15.07.2026.